

Week 3: Recogniser Implementation

In this week's lab, you are asked to implement a recogniser for the Microbrew language. A recogniser is like a parser, but it only goes as far as giving you a yes/no answer about whether a given input string is a valid program, it doesn't construct a structured representation of the program such as an abstract syntax tree.

Consequently, the implementation is a bit simpler than a full parser, although the structure is identical. In particular, you will need to start by implementing a lexer to tokenise the input, then implementation of the recogniser component follows the same recipe as the implementation of a predictive parser from an LL(1) parse table - just without the AST construction.

Feel free to implement the recogniser in whichever programming language you like, using whichever development environment you like. You may consult the reference implementation of the [Microbrew Interpreter](#), which includes a parser written in OCaml. Also, some hints are provided for implementation in Java, which easily generalise to any object-oriented programming language.

1. Define a datatype for lexer tokens. There should be one kind of token for each of the eight terminal symbols that make up the lexical structure of Microbrew:

var ident () , def = \$

When implementing a parser, the variable and identifier tokens would have to carry some extra data, the lexeme, so that we can name variables and identifiers correctly when we construct the abstract syntax tree. Here, since we are implementing a recogniser, we don't need the lexemes.

In Java, one way to do this is to define an enumeration of the possible terminal symbols:

```
public enum Token {
    IDENT,
    VAR,
    LPAREN,
    RPAREN,
    DEFINE,
    COMMA,
    EQUALS,
    END
}
```

2. Implement the lexer internal state, which represents a cursor into the input string, initialised to position 0, and a buffer for the output sequence of tokens, initialised to the empty sequence.
 - peek() which returns either the character under the cursor, or a signal to indicate that the

end of the input string has been reached.

- `drop()` which discards the character under the cursor and moves the cursor one position to the right.
- `emit(tk)` which appends the token *tk* to the output buffer.

In Java, this can be done with three fields, using an `ArrayList` to represent the sequence of tokens.

```
public final class Lexer {  
  
    private int idx = 0;  
    private String input;  
    private final List<Token> output = new ArrayList<>();  
  
    private Character peek() {  
        :  
    }  
  
    private void drop() {  
        :  
    }  
  
    private void emit(Token tk) {  
        :  
    }  
}
```

3. Implement the recursive lexing functions, whose behaviour is described by the following pseudocode. First the lexing function corresponding to the lexer initial state.

```
lex_init():  
    if peek() signals the end of the string then  
        emit the end-of-input token and return  
    else  
        let c be the character returned by peek()  
        if c is '(', ')', '=', ',', then  
            drop(); emit the corresponding token; lex_init()  
        elseif c is a whitespace character '\n', '\r' or ' ' then  
            drop(); lex_init()  
        elseif c is a lower or upper case letter then  
            lex_var_or_id_or_kw("")  
        else  
            FAIL
```

Second, the lexing function corresponding to the state in which the lexer is scanning a variable, identifier, or keyword (with substring scanned so far being the string parameter *lexeme*).

```

lex_var_or_id_or_kw(lexeme):
    let c be the character returned by peek()
    if c is a lower or uppercase letter then
        let lexeme' be lexeme with c appended to the end
        drop(); lex_var_or_id_or_kw(lexeme')
    else
        if lexeme = "def" then
            emit the define token
        elseif the first character of lexeme is lowercase then
            emit the variable token
        else
            emit the identifier token
lex_init()

```

4. Implement the lexer main function `lex(s)` which takes as input a string `s` to be lexed and returns the corresponding sequence of tokens. Your implementation should pass the following tests, in which `lp` stands for the left parenthesis token and `rp` for the right parenthesis token:

Input	Output
<code>Z()</code>	<code>ident,lp,rp,end</code>
<code>def One() = S(Z())</code>	<code>define,ident,lp,rp,equal,ident,lp,ident,lp,rp,rp,end</code>
<code>def Head(C(x,xs)) = x</code>	<code>define,ident,lp,ident,lp,var,comma,var,rp,rp,equal,var,end</code>
<code>Add(S(Z()), Z())</code>	<code>ident,lp,ident,lp,ident,lp,rp,rp,comma,ident,lp,rp,rp,end</code>

If you like, you can construct more test cases by extracting lines from the test cases supplied with the Microbrew reference interpreter: [Booleans](#), [Numbers](#), [Lists](#).

5. Implement the recogniser internal state, which consists of the list of so-far unconsumed tokens.
- `peek()` returns the next unconsumed token.
 - `eat(tk)` consumes the next token only if it is precisely token `tk` and throws an exception otherwise.
6. Implement the parsing (recognising) functions according to the standard recipe for recursive descent predictive parsing based on an LL(1) parsing table:
- Define one parsing function for each nonterminal (row) `N`. Each parsing function takes no input and returns no output but will fail as a side-effect (throw an exception) if parse failure is detected.
 - Each parsing function begins by inspecting the next unconsumed token, say `tk`.
 - The cell corresponding to row `N` and column `tk` describes how the parser (recogniser) should behave. The cell is either empty, in which the parser should fail by throwing an exception, or it contains (the right-hand side of) exactly one grammar rule for nonterminal `N`. The right-hand side of a grammar rule is a sentential form, and the parser should proceed to perform the actions specified by the symbols in this sentential form, from left-to-right.

	\$	def	ident	(	)	=	var	,
<i>Cmd</i>		def ident (<i>ExpList</i>) = <i>Exp</i> end	<i>Exp</i> end				<i>Exp</i> end	
<i>Exp</i>			ident (<i>ExpList</i>)				var	
<i>ExpList</i>			<i>Exp ExpList'</i>		ε		<i>Exp ExpList'</i>	
<i>ExpList'</i>					ε			, <i>Exp ExpList</i>

Table 1: Parsing Table for Microbrew

- If the symbol is a terminal, the parser should eat it (i.e. check that the next unconsumed token is this terminal and consume).
- If the symbol is a nonterminal, the parser should call the corresponding parsing function.

The parsing table for Microbrew is given in Table 1.

In Java, the parsing function for *Exp* can be implemented as follows:

```
private void rExp() {
    switch (peek()) {
        case IDENT -> {
            eat(Token.IDENT);
            eat(Token.LPAREN);
            rExpList();
            eat(Token.RPAREN);
        }
        case VAR -> eat(Token.VAR);
        default -> throw new ParseError();
    }
}
```

7. Implement a top-level parse function which takes a string *s* as input:

1. Calls on the lexer to transform it to a sequence of tokens *tk*s.
2. Initialises the recogniser internal state with *tk*s.
3. Calls on the parsing (recognising) function corresponding to the start nonterminal *Cmd*.
4. If this call returns (void), return true. If this call fails with an exception, catch it and return false.

Ensure that your recogniser passes the following tests:

Input	Output	Input	Output
ε	true	Z()Z	false
Add(Z(),S(Z()))	true	Add(Z(),S(Z()	false
def Add(Z(),y) = y	true	def Add(Z(),y)	false
def Head(C(x,xs)) = x	true	Z!	false